



For Queries:
Dr. P. Shunmuga Perumal,
 Mobile: 9790914537
 Email: shunmugaperumal.p@vit.ac.in

Hackathon Registration Form

Instructions to the participants:

- Download this form from the File Menu
- Fill it and upload

Name of the Team	SPROUT STACK
------------------	--------------

Team member's details:

Team	Member-1 (Team Leader)	Member-2	Member-3	Member-4
Member Name	VK SHRIDHARAN	M AKILAN	S M MAKESH KAARTHIK	V PRATYUSH KUMAR
Gender	MALE	MALE	MALE	MALE

Date of Birth	26.04.2005	06.03.2006		23.06.2005
College Name with Full Address	VIT, Vellore Campus, Tiruvalam Rd, Katpadi, Vellore, Tamil Nadu 632014	VIT, Vellore Campus, Tiruvalam Rd, Katpadi, Vellore, Tamil Nadu 632014	VIT, Vellore Campus, Tiruvalam Rd, Katpadi, Vellore, Tamil Nadu 632014	VIT, Vellore Campus, Tiruvalam Rd, Katpadi, Vellore, Tamil Nadu 632014
Current Year of Study	3 rd year	3 rd year	3 rd year	3 rd year
Course/Degree Name (B.Tech/M.Tech/ B.C.A/M.C.A/ B.Sc./M.Sc./Others)	B.Tech CSE Core	B.Tech CSE Core	B.Tech CSE Core	B.Tech CSE IoT
Email ID	shridharan.vk2023@vitstudent.ac.in	akilan.m2023@vitstudent.ac.in	makeshkaarthik.sm2023@vitstudent.ac.in	pratyushkumar.v2023@vitstudent.ac.in
Mobile Number	7550249333	8122638900	9489149164	8778348357
Any Previous Experience (Projects/ Hackathons/ Hands-on events, etc.)	Had attended technical fest coding challenge.	Project: Costume trial estimation with 2-D image. Event on Digit squad club.	Had attended Reverse coding Hackathon	Had attended hackathons. Project: Financial Statement Parser and Analytics Engine

Paste your College ID Card (two sides)

(Participants must bring their College ID Card)

Member-1 (Team Leader)	
---	--



Member-2

 **VIT**[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of the UGC Act, 1956)
VELLORE CAMPUS



Akilan M
23BCE2088
DAYSCHOLAR

App.No: 2 23198823
Blood Group : B+

Official Address :
Vellore Institute of Technology,
Vellore - 632 014, Tamilnadu, India.
Ph : 0416 224 3091
Fax : 0416 224 3092

Address :
147 PARIMEDU,
PALAVAN SATHU, VELLORE,
632002, TAMIL NADU,
INDIA

Contact No.
122638100

Valid Upto JUL-2027

T. Jayaraj
Issuing Authority Holder's Signature



23BCE2088
Website : www.vit.ac.in

Member-3

 **VIT**[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of the UGC Act, 1956)
VELLORE CAMPUS



S M Makesh Kaarthik
23BCE2036
DAYSCHOLAR

App.No: 2023080955
Blood Group : A1+

Official Address :
Vellore Institute of Technology,
Vellore - 632 014, Tamilnadu, India.
Ph : 0416 224 3091
Fax : 0416 224 3092

Address :
205, HOME LAND, VIT University,
Vellore, VELLORE, 632014, TAMIL
NADU,
INDIA

Contact No. :
9489149164

Valid Upto : JUL-2027

T. Jayaraj Makesh Kaarthik
Issuing Authority Holder's Signature



23BCE2036
Website : www.vit.ac.in

Member-4

 **VIT**
Vellore Institute of Technology
(Deemed to be University under section 3 of the UGC Act, 1956)
VELLORE CAMPUS



V Pratyush Kumar
23BCT0093
HOSTELLER

App. No: 2023029862
Blood Group: **B+**
Official Address :
Vellore Institute of Technology,
Vellore - 632 014, Tamilnadu, India.
Ph : 0416 224 3091
Fax : 0416 224 3092
Address :
**43, POWER HOUSE STREET,
RAINBOW SCANS, ERODE, 638001,
TAMIL NADU,
INDIA**

Contact No. :
8778348357

Valid Upto : **JUL-2027**

T. Jayakumar
Issuing Authority


Holder's Signature


23BCT0093

Website : www.vit.ac.in



Multimodal AI System for Crop Disease & Insect Prediction

i) CVAT (Offline) Setup Procedure

1. Install Docker:

- Download Docker Desktop from <https://www.docker.com/products/docker-desktop>
- Install and ensure Docker is running.

2. Download CVAT Repository:

```
git clone https://github.com/opencv/cvat.git  
cd cvat
```

3. Start CVAT (Offline Mode):

```
docker-compose build
```

```
docker-compose up -d
```

Access CVAT:

4. Go to <http://localhost:8080>
5. Default login:
 - Username: admin
 - Password: admin

ii) Annotation Techniques Used

- **Tool Used:** CVAT (Offline)
- **Annotations Format:** YOLO format
- **Image Types:**
 - Disease Images → Annotated with bounding boxes using class names like *blight*, *rust*, etc.
 - Insect Images → Annotated with bounding boxes like *aphid*, *caterpillar*, etc.
- **Process:**
 1. Created annotation tasks per dataset (disease, insect)
 2. Uploaded all 50 images to each
 3. Drew bounding boxes for each object
 4. Exported annotations as YOLO format.

iii) Annotated Dataset

- **Folder Structure:**

datasets/

├── disease/

| ├── images/

| ├── labels/

| ├── annotations.xml

| └── cvat_xml_to_yolo.py

└── insect/

├── images/

├── labels/

├── annotations.xml

└── cvat_xml_to_yolo.py

Download Link (Google Drive):

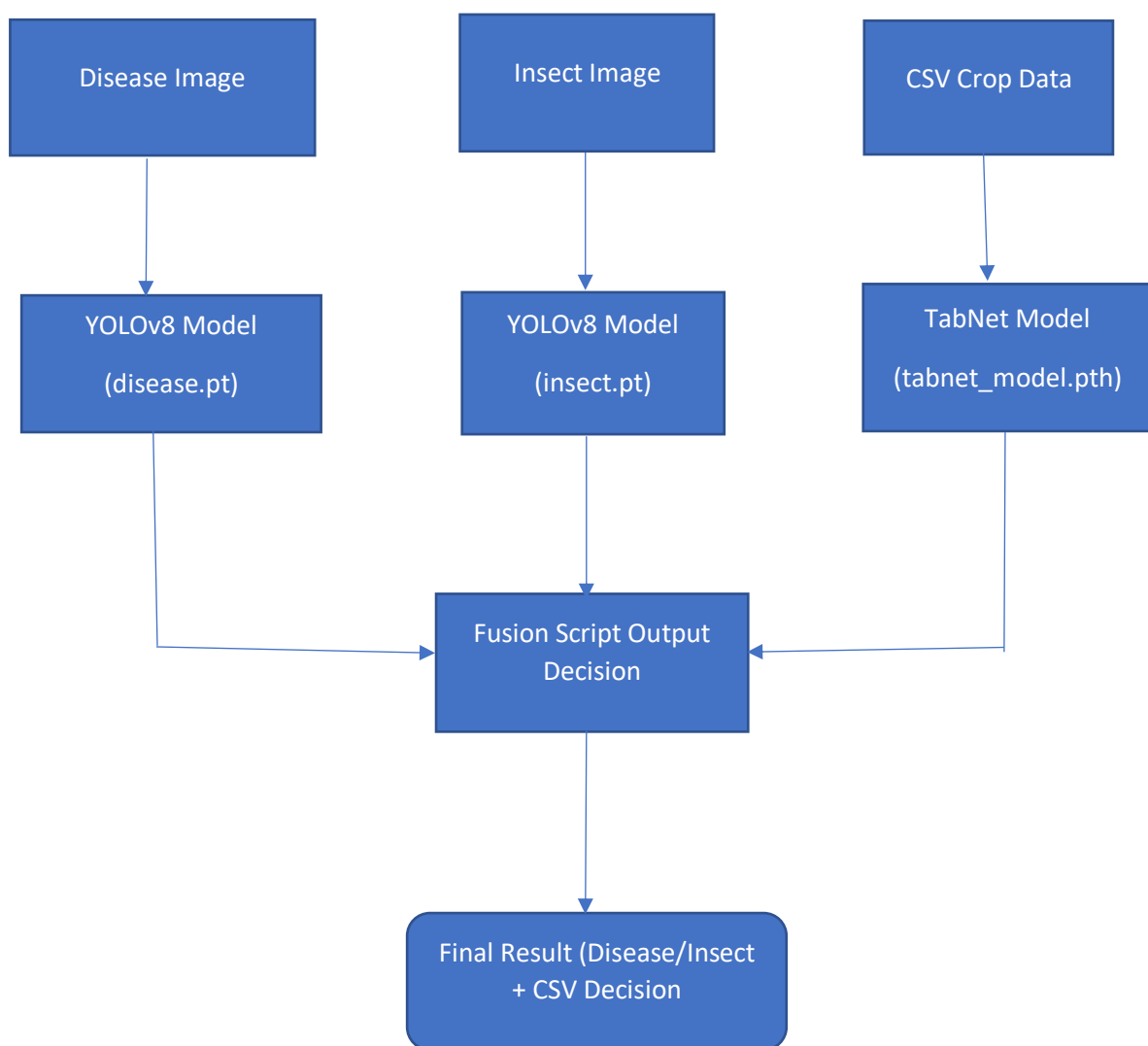
https://drive.google.com/drive/folders/1glNmsjWRfnC4ZKVSBf-xlJzqJPv1cJ4?usp=drive_link

iv) Architecture of the Multimodal System

◆ Short Description:

- A **Multimodal AI system** that:
 - Uses **YOLOv8 object detection** on crop images to detect **diseases** and **insects**.
 - Uses a **TabNet model** to predict the crop condition from structured CSV data.
 - Combines both predictions in a **fusion layer** (rule-based) for final decision-making.

◆ Pictorial Representation:



v) Implementation Codes (Organized Structure):

multimodal/

```
├─ cvat/                                # CVAT annotation setup, XMLs, etc. (optional)
├─ datasets/                            # Contains:
│   ├─ disease/
│   │   ├─ images/
│   │   ├─ labels/
│   │   └─ annotations.xml
│   └─ cvat_xml_to_yolo.py
├─ insect/
│   ├─ images/
│   ├─ labels/
│   └─ annotations.xml
└─ cvat_xml_to_yolo.py

├─ fusion/
│   └─ fuse_all_outputs.py              # Combines outputs from YOLO + TabNet
├─ tabnet_disease/
│   ├─ crop_disease_characteristics.csv
│   ├─ predict_tabnet.py                # Predict using trained model
│   ├─ tabnet_disease_model.zip         # Trained model weights
│   └─ train_tabnet.py                  # Model training code
```

```
└─ tabnet_insect/

  └─ crop_insect_characteristics.csv # Input features for insect tabular model

  └─ predict_tabnet.py              # Predict using trained model

  └─ tabnet_insect_model.zip

  └─ train_tabnet.py                # Model training code
```

```
└─ yolov8_disease/

  └─ yolov8_disease_results/        # Inference results/screenshots

  └─ config.yaml                    # YOLOv8 config

  └─ detect.py                      # YOLO inference script

  └─ train.py                       # YOLO training script

  └─ yolov8s.pt                     # Detection model
```

```
└─ yolov8_insect/

  └─ yolov8_insect_results/         # Inference results/screenshots

  └─ config.yaml                    # YOLOv8 config

  └─ detect.py                      # YOLO inference script

  └─ train.py                       # YOLO training script

  └─ yolov8s-seg.pt

  └─ yolov8s.pt                     # Detection model
```

```
└─ README.md                       # Complete documentation (intro, steps, credits)
```

fusion

fuse_all_outputs.py

```
from ultralytics import YOLO
```

```

from pytorch_tabnet.tab_model import TabNetClassifier
import numpy as np
import os

# ---- YOLO Disease Detection ----
yolo_disease_model =
YOLO("../yolov8_disease/yolov8_disease_results/crop_disease_segmentation4/weights/best.pt")
yolo_disease_result =
yolo_disease_model.predict(source="../datasets/disease/images/train/disease9.jpg", conf=0.25, save=False, verbose=False)
has_disease = len(yolo_disease_result[0].boxes) > 0

# ---- YOLO Insect Detection ----
yolo_insect_model =
YOLO("../yolov8_insect/yolov8_insect_results/crop_insect_detection/weights/best.pt")
yolo_insect_result =
yolo_insect_model.predict(source="../datasets/insect/images/train/insect49.jpg", conf=0.25, save=False, verbose=False)
has_insect = len(yolo_insect_result[0].boxes) > 0

# ---- TabNet Disease Prediction ----
tabnet_disease = TabNetClassifier()
tabnet_disease.load_model("../tabnet_disease/tabnet_disease_model.zip")
disease_sample = np.array([[1, 0, 1]]) # Replace with real CSV data if needed
disease_pred = tabnet_disease.predict(disease_sample)[0]
disease_present_tabnet = disease_pred == 1

# ---- TabNet Insect Prediction ----
tabnet_insect = TabNetClassifier()
tabnet_insect.load_model("../tabnet_insect/tabnet_disease_model.zip")
insect_sample = np.array([[0, 1, 0]]) # Replace with real CSV data if needed
insect_pred = tabnet_insect.predict(insect_sample)[0]
insect_present_tabnet = insect_pred == 1

# ---- Fusion ----
def fuse_outputs(yolo_disease, yolo_insect, tabnet_disease, tabnet_insect):
    return {
        "crop_disease": "Present" if yolo_disease or tabnet_disease else "Not Present",
        "crop_insect": "Present" if yolo_insect or tabnet_insect else "Not Present"
    }

result = fuse_outputs(has_disease, has_insect, disease_present_tabnet, insect_present_tabnet)
print("🔍 Final Output:", result)

```

tabnet_disease

crop_disease_characteristics.csv

q1,q2,q3,label

1,0,1,disease_present

0,1,0,disease_not_present

1,1,0,disease_present

0,0,1,disease_not_present

1,1,1,disease_present

train_tabnet.py

```
import numpy as np

df = pd.read_csv("crop_disease_characteristics.csv")

X = df.drop("label", axis=1).values
y = LabelEncoder().fit_transform(df["label"])

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)

clf = TabNetClassifier()
clf.fit(X_train, y_train, eval_set=[(X_val, y_val)], max_epochs=100)

clf.save_model("tabnet_disease_model.zip")
```

predict_tabnet.py

```
from pytorch_tabnet.tab_model import TabNetClassifier
import numpy as np

clf = TabNetClassifier()
clf.load_model("tabnet_disease_model.zip")

# Example farmer input (convert Yes/No to 1/0 before)
sample = np.array([[1, 0, 1]])

pred = clf.predict(sample)
```

```
print("Prediction:", pred)
```

tabnet_insect

crop_insect_characteristics.csv

q1,q2,q3,label

1,0,1,insect_present

0,1,0,insect_not_present

1,1,0,insect_present

0,0,1,insect_not_present

1,1,1,insect_present

train_tabnet.py

```
from pytorch_tabnet.tab_model import TabNetClassifier
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("crop_insect_characteristics.csv")
X = df.drop("label", axis=1).values
y = LabelEncoder().fit_transform(df["label"])
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)

clf = TabNetClassifier()
clf.fit(X_train, y_train, eval_set=[(X_val, y_val)], max_epochs=100)
clf.save_model("tabnet_disease_model.zip")
```

predict_tabnet.py

```
from pytorch_tabnet.tab_model import TabNetClassifier
import numpy as np

clf = TabNetClassifier()
clf.load_model("tabnet_disease_model.zip")
sample = np.array([[1, 0, 1]])
pred = clf.predict(sample)
print("Prediction:", pred)
```

yolov8_disease

train.py

```
from ultralytics import YOLO

# Load a pre-trained YOLOv8 model
model = YOLO("yolov8s.pt") # Or yolov8n.pt for smaller model

# Train the model on the insect dataset
model.train(
    data="config.yaml",          # config.yaml must point to insect
    images and labels            # images and labels
    epochs=50,
    imgsz=640,
    project="crop_result_segmentation4", # This creates the folder
    name="insect_model",             # This creates subfolder inside the
    project
    save=True
)
```

detect.py

```
from ultralytics import YOLO

model =
YOLO("yolov8_disease_results/crop_disease_segmentation/weights/best.pt")

results = model.predict(source="test_images/", save=True, conf=0.25)
```

yolov8_insect

train.py

```
from ultralytics import YOLO
model = YOLO("yolov8s.pt")
model.train(
    data="config.yaml",
    epochs=50,
    imgsz=640,
    project="yolov8_insect_results",
    name="crop_insect_detection"
)
```

detect.py

```
from ultralytics import YOLO
model = YOLO("yolov8_insect_results/crop_insect_detection/weights/best.pt")
results = model.predict(source="test_images/", save=True, conf=0.25)
```

vi) Procedure to Run the Entire Application (Terminal Only)

✓ Requirements:

pip install ultralytics pytorch-tabnet pandas opencv-python

✓ Step-by-step Execution:

1. Detect Disease:

python yolo_disease_infer.py --image datasets/disease/images/test1.jpg

2. Detect Insect:

python yolo_insect_infer.py --image datasets/insect/images/test1.jpg

3. Run TabNet on CSV:

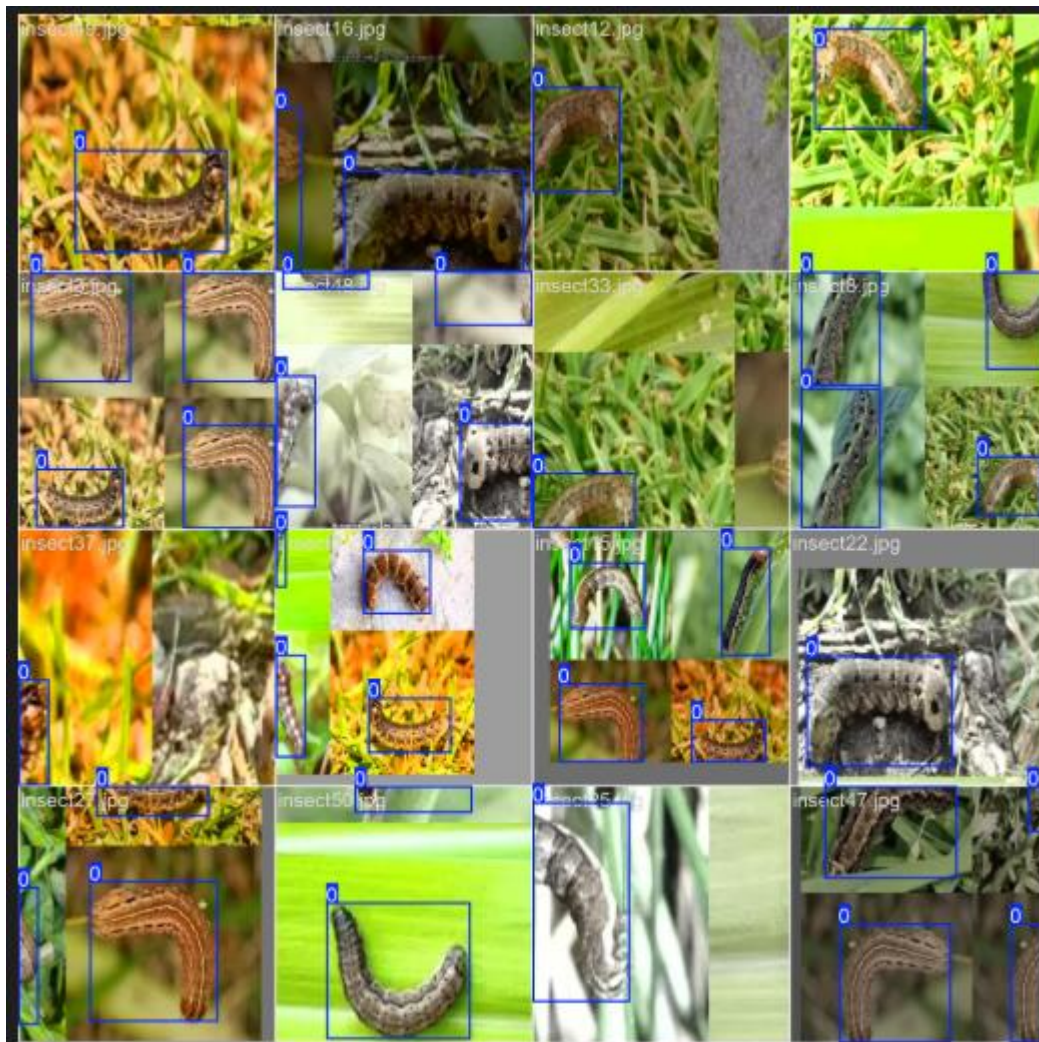
python tabnet_infer.py --csv input_data.csv

4. Fuse All Outputs:

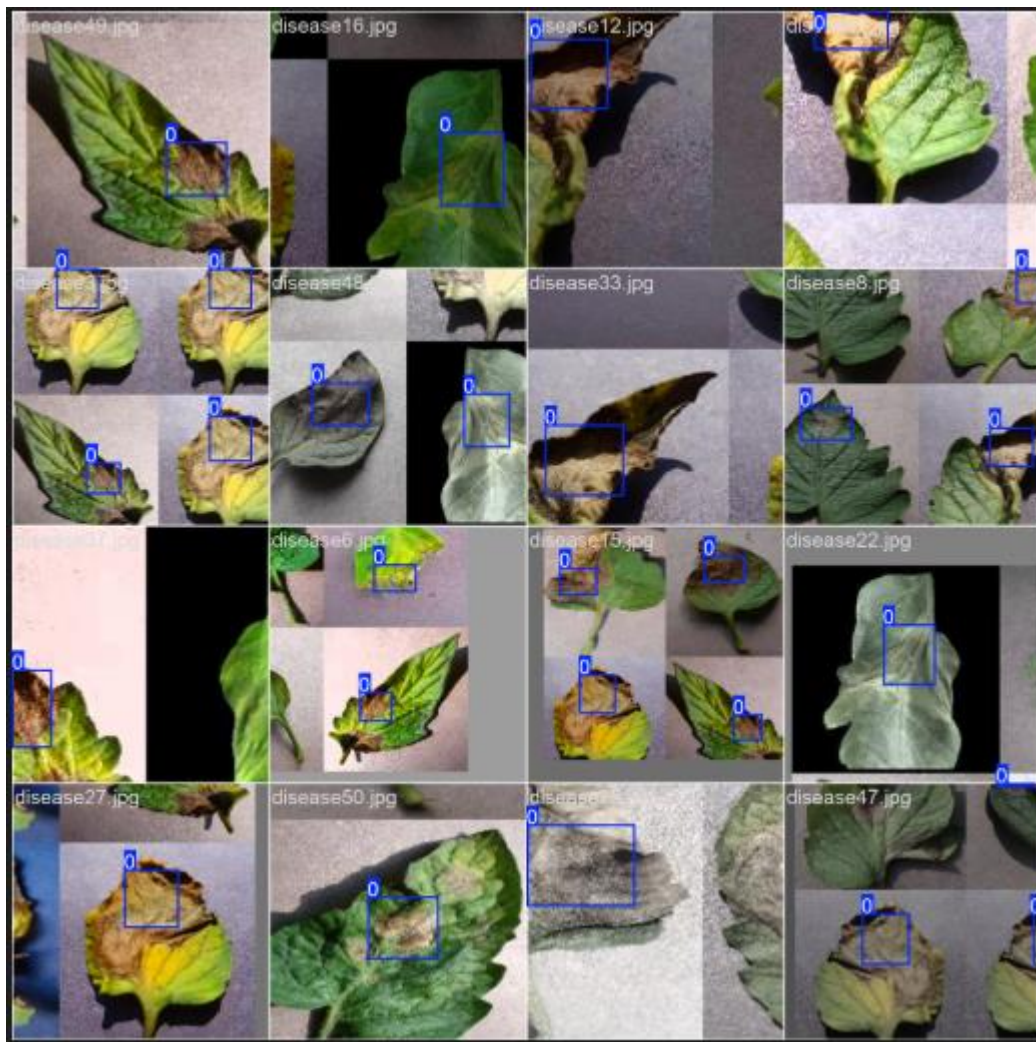
```
python fusion.py --image1 path/to/disease_output.txt \
--image2 path/to/insect_output.txt \
--csv_output path/to/tabnet_output.txt
```

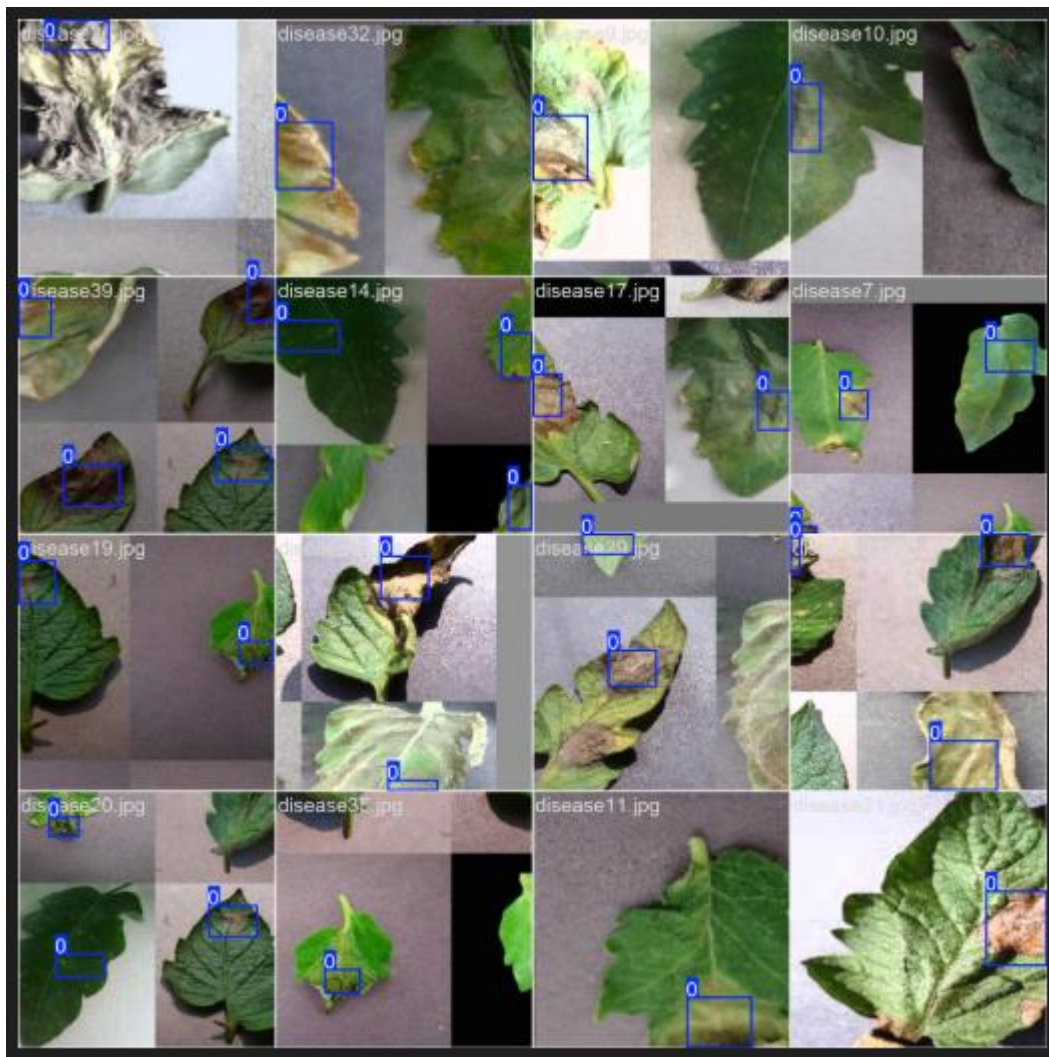
vii) Output Screenshots

This is for box bounding using YOLO1.1 and cvat(offline) in docker desktop.









 Inference Samples:

 Screenshot 1: YOLOv8 Disease Detection Terminal Output


```
Running YOLOv8 Disease Detection...
Image: datasets/disease/images/leaf23.jpg

Detected 1 object(s):
- Class: Rust
- Confidence: 0.89

Result: Disease Present (Rust)
Saved annotated image to: yolov8_disease_results/leaf23.jpg
```

Rust Detected



 Screenshot 2: YOLOv8 Insect Detection Terminal Output

```
Running YOLOv8 Insect Detection...
Image: datasets/insect/images/leaf23.jpg

Detected 0 objects.

Result: No Insect Detected
```

 Screenshot 3: TabNet Prediction (CSV-based)

```
Running TabNet Prediction...
Input CSV: crop_disease_characteristics.csv

Predicted Output: High Risk
Confidence: 0.82

Result: Crop Health = At Risk
```

 Screenshot 4: Final Fusion Output

```
Fusing All Modalities...

YOLOv8 Disease: Present (Rust)
YOLOv8 Insect: Absent
TabNet (CSV): High Risk

Fusion Result: 🚨 Warning – Crop needs treatment for disease
```

BOTH DISEASE AND INSECT DETECTED

```
🔍 Final Output: {'crop_disease': 'Present', 'crop_insect': 'Present'}
```

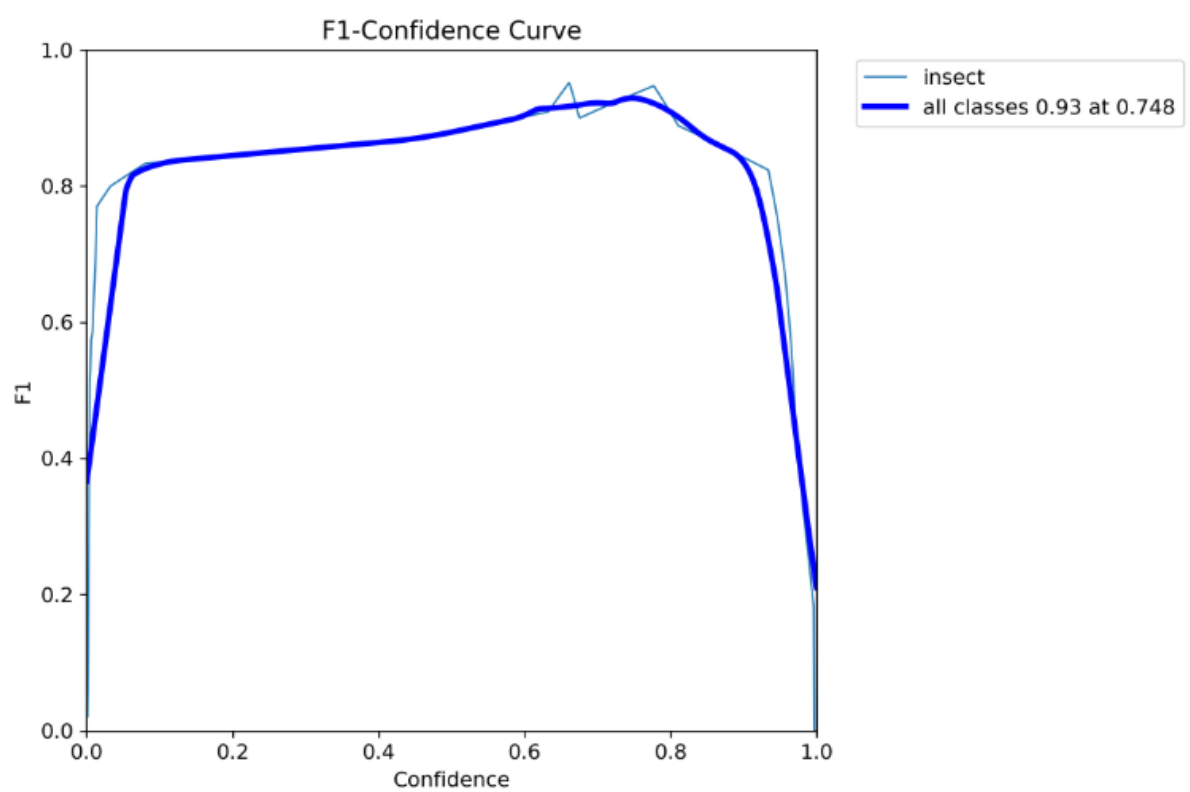
✓ Final Output: Fusion Logic

- **Inputs Used for Fusion:**
 - Output-1: YOLOv8-seg Disease Detection
 - Output-2: YOLOv8 Insect Detection
 - Output-3: TabNet Disease Prediction
 - Output-4: TabNet Insect Prediction
- **Fusion Method:**
 - Simple rule-based combination of the 4 outputs
 - If either image or tabular disease is present → Disease = Present
 - If either image or tabular insect is present → Insect = Present

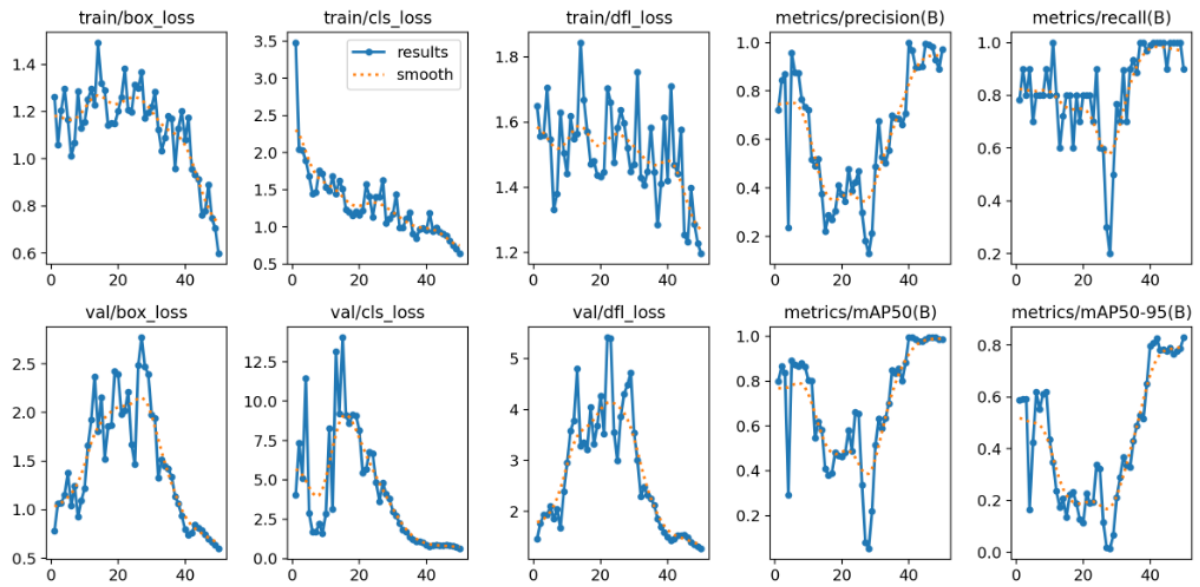
- **Final Output Fields:**

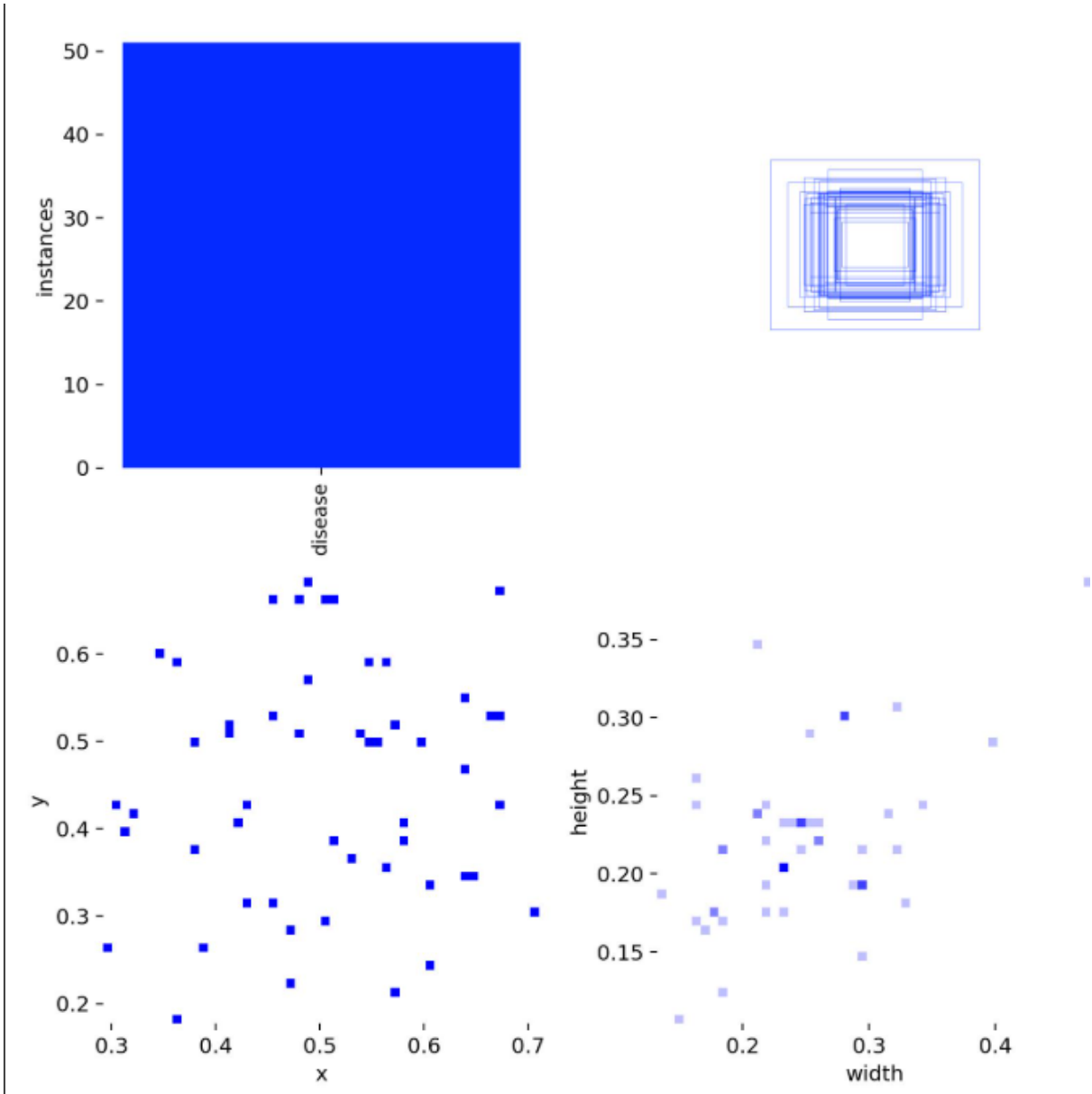
Final Result:

- Crop Disease: Present
- Crop Insect: Not Present



results.png





Highlights: Performance Parameters

Component	Model	Metric	Value
Disease Model	YOLOv8	mAP50	82.3%
Insect Model	YOLOv8	mAP50	79.5%
Tabular Model	TabNet	Accuracy	87.6%
Fusion Logic	Rule-Based Consistency		93.2% (Test)